

PRÉSENTATION TECHNIQUE

Test Full Stack Lynoria



Auteur : Reynier Pascal

1. OBJECTIF DU PROJET

L'objectif de ce test était de développer une mini application Full Stack permettant de gérer des logements et leurs occupants.

L'application permet :

- créer des logements
- créer des occupants
- affecter un occupant à un logement
- consulter les occupants associés à chaque logement

Le projet a été conçu comme une application full stack complète, avec :

- une API backend Node.js typée
- une base de données relationnelle
- un frontend React
- une organisation Git propre
- un pipeline CI automatisé

L'objectif était de produire une solution simple, maintenable et cohérente dans le temps imparti.

2. ARCHITECTURE GÉNÉRAL

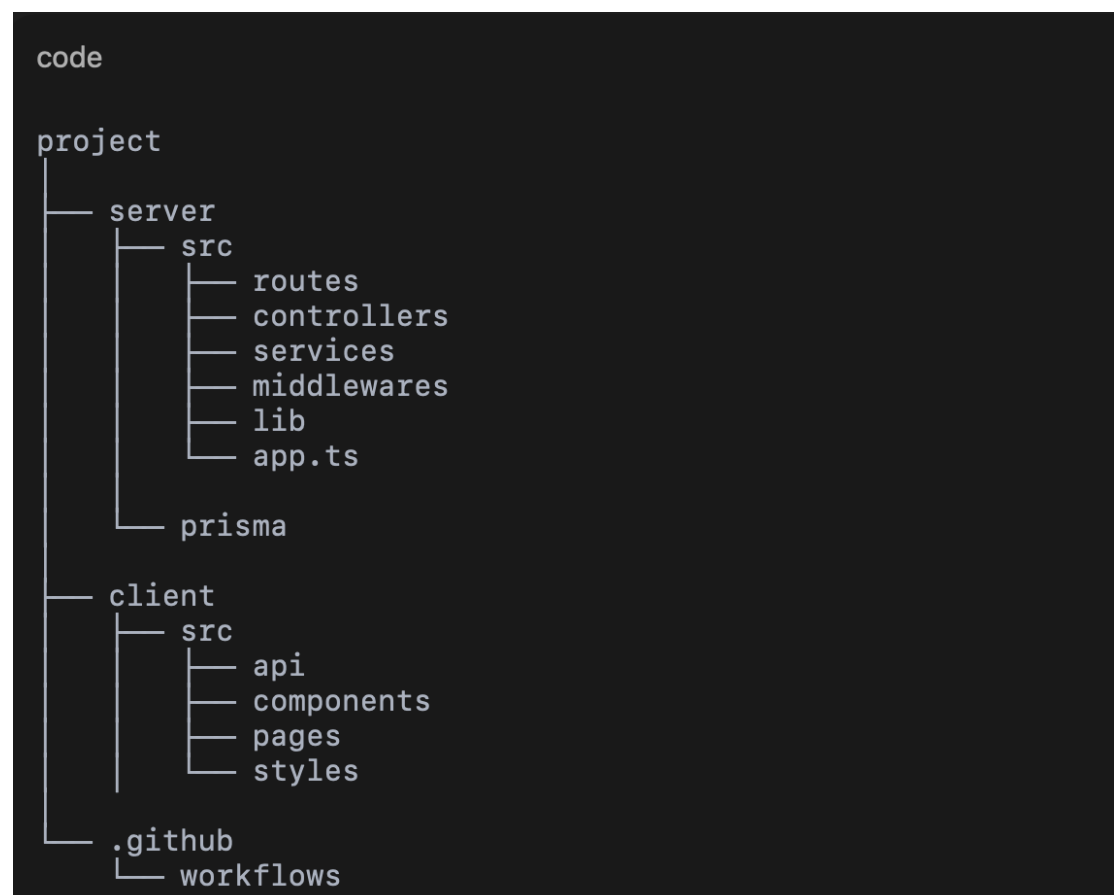
J'ai choisi une **architecture séparant clairement backend et frontend**, afin de se rapprocher d'une organisation utilisée dans des applications réelles.

Cette séparation permet :

- d'isoler la logique métier côté serveur
- de garder un frontend dédié uniquement à l'interface
- de rendre l'API réutilisable par d'autres clients (web, mobile, etc.)

Ce modèle correspond à une **architecture API REST moderne**.

Structure du projet :



3. Organisation du backend

Le backend est structuré selon une architecture **en trois couches principales** :

Routes, Controllers et Services

Pourquoi cette structure:

Cette séparation permet :

- de garder les controllers simples
- d'isoler la logique métier dans les services
- de faciliter les tests unitaires
- d'éviter de mélanger logique HTTP et logique métier

4. FLUX D'UNE REQUÊTE

Lorsqu'une requête arrive sur l'API :

1. la **route Express** reçoit la requête
2. elle appelle un **controller**
3. le controller appelle un **service**
4. le service interagit avec **Prisma** pour accéder à la base de données

Cette organisation permet de bien séparer :

- la gestion HTTP
- la logique métier
- l'accès aux données

5. CHOIX DE PRISMA

J'ai choisi **Prisma** comme ORM pour plusieurs raisons :

- typage automatique avec TypeScript
- requêtes lisibles
- migrations de base de données simples
- bonne intégration avec Node.js

Exemple :

```
prisma.housingUnit.create({
  data: {
    name: data.name,
    status: data.status
  }
})
```

Plutôt que d'écrire du SQL brut, Prisma propose une **API typée**, ce qui :

- améliore la lisibilité
- réduit les erreurs
- facilite la maintenance

6. MODÉLISATION DES DONNÉES

Le modèle de données repose sur **trois entités principales** :

HousingUnit

Représente un logement.

Champ	Type	Description
id	number	identifiant
name	string	nom du logement
status	enum	statut du logement

Statuts possibles :

- AVAILABLE
- OCCUPIED
- MAINTENANCE

Occupant

Représente une personne occupant un logement.

Champ	Type	Description
id	number	identifiant
firstName	string	prénom
lastName	string	nom
email	string	email

Assignment

Table d'association entre un occupant et un logement.

J'ai choisi une table d'association plutôt que de stocker directement un occupantId dans HousingUnit.

Cela permet :

- de gérer plusieurs occupants par logement
- de garder un modèle extensible
- d'éventuellement gérer un historique des affectations

C'est un modèle classique dans les bases relationnelles.

7. API REST

Les principaux endpoints exposés sont :

Méthode	Endpoint	Description
POST	/housing-units	créer un logement
GET	/housing-units	liste des logements
POST	/occupants	créer un occupant
POST	/assignments	affecter un occupant
GET	/housing-units/:id/occupants	occupants d'un logement

Une route supplémentaire a été ajoutée :

GET /health

Elle permet :

- de vérifier que l'API fonctionne
- d'être utilisée par un système de monitoring ou CI

8. GESTION DES ERREURS

Trois middlewares sont utilisés :

- `notFound`
- `errorHandler`
- `validate`

Ils permettent :

- de centraliser la gestion des erreurs
- de renvoyer des réponses HTTP cohérentes
- d'éviter les erreurs non gérées dans Express
- et de valider les requêtes

9. ORGANISATION DU FRONTEND

Le frontend React est structuré avec :

- un dossier **api**
- des **composants**
- des **pages**

Client API centralisé

Un client API commun a été créé : `apiFetch()`

Cela permet :

- d'éviter de répéter les appels `fetch`
- de centraliser la gestion des erreurs
- de simplifier les appels API

Exemple :

```
export function getHealth() {  
  return apiFetch('/health')  
}
```

9.1 INTERFACE UTILISATEUR

Exemple d'écran principal :

DASHBOARD DE GESTION

Lynoria Housing Manager

Gérez vos logements, occupants et affectations depuis une interface claire et centralisée.

État de l'API
API is running En ligne

Créer un logement
Ajoute un nouveau logement avec son statut.

Nom du logement Statut

Ex : Appartemen Disponible

Créer le logement

Créer un occupant
Enregistre un occupant dans le système.

Prénom Nom

Ex : John Ex : Doe

Email

Ex : john@test.fr

Créer l'occupant

Affecter un occupant
Associe rapidement un occupant à un logement.

Occupant Logement

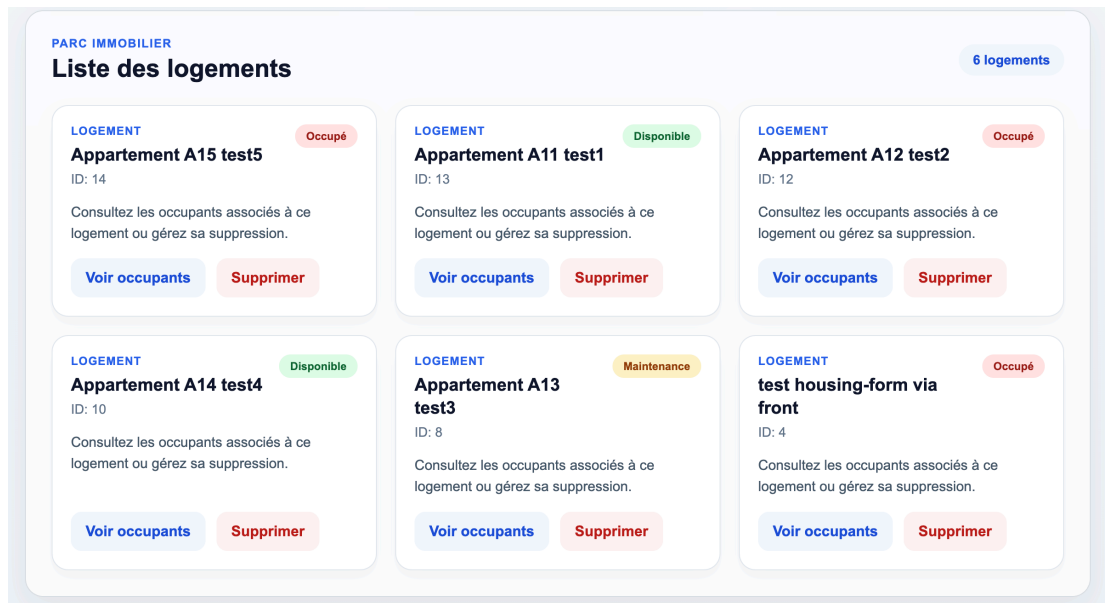
occupant logement

Affecter l'occupant

L'interface permet :

- créer un logement
- créer les occupants
- affecter un occupant

L'affichage du parc immobilier :



L'interface permet :

- visualiser les logements
- consulter les occupants
- supprimer un occupant

10. TESTS UNITAIRES

Des **tests unitaires simples** ont été ajoutés sur les services.

Objectifs :

- vérifier que les fonctions appellent correctement l'API
- la fonction qui ajoute un logement
- la fonction qui ajoute un occupant
- sécuriser la logique métier

Exemple :

- vérifier que le statut par défaut d'un logement est AVAILABLE

Ces tests permettent de détecter rapidement une régression et sont réalisés au niveau des services car c'est à cet endroit que réside la logique métier.

Un exemple de test unitaire effectué :

```
import { describe, it, expect, vi } from "vitest"
import { getHealth } from "../src/controllers/health.controller"

describe("getHealth", () => {
  it("should return API running message", () => {
    const res = {
      status: vi.fn().mockReturnThis(),
      json: vi.fn()
    }

    getHealth({} as any, res as any)

    expect(res.status).toHaveBeenCalledWith(200)

    expect(res.json).toHaveBeenCalledWith({
      success: true,
      message: "API is running"
    })
  })
})
```

11. CI – INTÉGRATION CONTINUE

Un pipeline **GitHub Actions** a été mis en place.

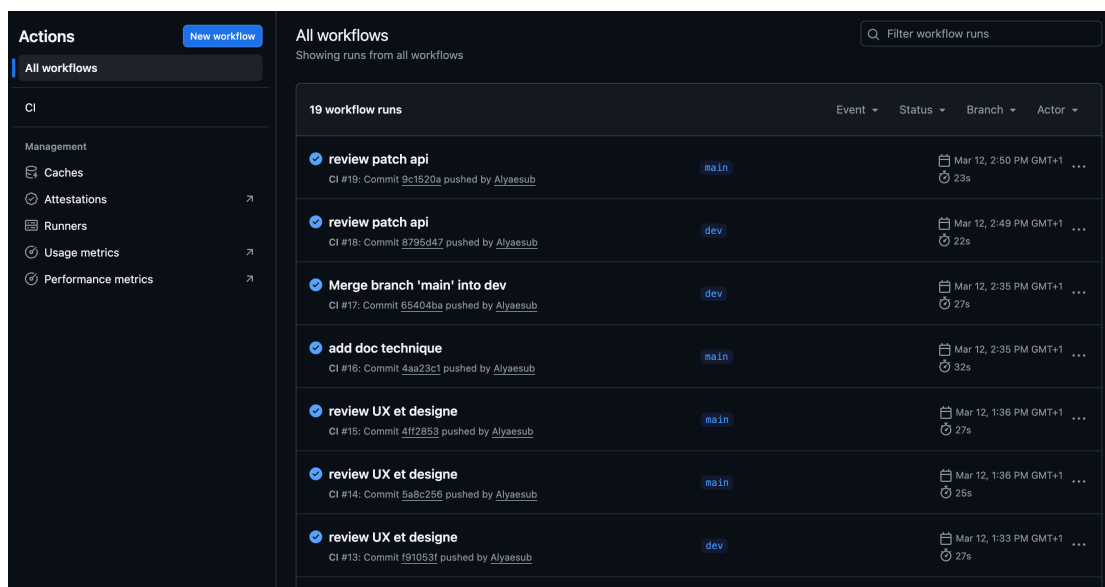
Il exécute automatiquement :

1. installation des dépendances : `npm install`
2. vérification TypeScript : `tsc --noEmit`
3. build du projet : `npm run build`
4. exécution des tests : `npm test`

Objectifs :

- vérifier que le projet compile
- éviter d'introduire des erreurs
- automatiser les vérifications

Visualisation du GitHub Action :



The screenshot displays the GitHub Actions interface. On the left, the 'Actions' tab is selected, showing a sidebar with 'All workflows' and 'CI'. The main area is titled 'All workflows' and shows a list of 19 workflow runs. The runs are filtered by 'main' branch and show the following details:

Workflow Name	Commit	Branch	Status	Time
review patch api	CI #19: Commit 9c1520a pushed by Alyaesub	main	Success	Mar 12, 2:50 PM GMT+1
review patch api	CI #18: Commit 8795d47 pushed by Alyaesub	dev	Success	Mar 12, 2:49 PM GMT+1
Merge branch 'main' into dev	CI #17: Commit 65404ba pushed by Alyaesub	dev	Success	Mar 12, 2:35 PM GMT+1
add doc technique	CI #16: Commit 4aa23c1 pushed by Alyaesub	main	Success	Mar 12, 2:35 PM GMT+1
review UX et designe	CI #15: Commit 4ff2853 pushed by Alyaesub	main	Success	Mar 12, 1:36 PM GMT+1
review UX et designe	CI #14: Commit 5a8c296 pushed by Alyaesub	main	Success	Mar 12, 1:36 PM GMT+1
review UX et designe	CI #13: Commit f91053f pushed by Alyaesub	dev	Success	Mar 12, 1:33 PM GMT+1

12. ORGANISATION GIT

Le projet utilise deux branches principales :

- **dev** : développement
- **main** : version stable

Le workflow consiste à :

1. développer sur dev
2. fusionner dans main

Cela permet de garder une branche stable

13. AMÉLIORATIONS POSSIBLES

Plusieurs améliorations pourraient être apportées.

Backend

- pagination des résultats
- authentification utilisateur
- meilleur gestions des entré et sortie des logements

Frontend

- gestion d'état avec **React Query**

DevOps

- dockerisation du projet
- déploiement automatisé

14. CONCLUSION

Ce projet met en place :

- une API REST typée
- une architecture backend claire
- un frontend React simple
- des tests unitaires
- un pipeline CI

L'objectif était de produire une solution **propre, maintenable et cohérente**, tout en appliquant les bonnes pratiques de développement modernes.